Classes in C++

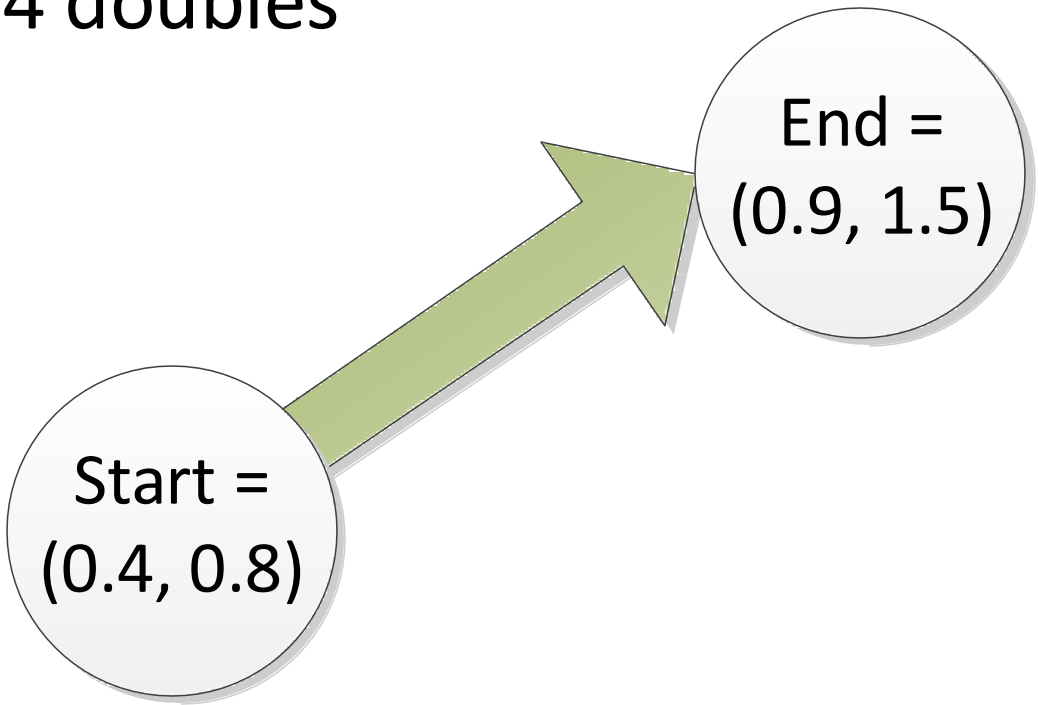
Representing a (Geometric) Vector

- In the context of geometry, a vector consists of 2 points: a start and a finish
- Each point itself has an x and y coordinate

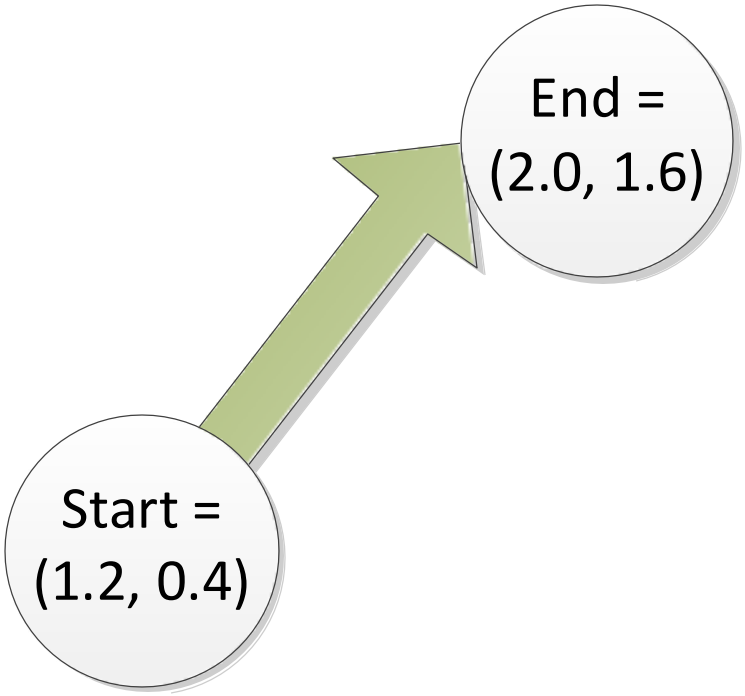


Representing a (Geometric) Vector

- Our representation so far? Use 4 doubles (startx, starty, endx, endy)
- We need to pass all 4 doubles to functions



```
int main() {
  double xStart = 1.2;
  double xEnd = 2.0;
  double yStart = 0.4;
  double yEnd = 1.6;
}
```



```
void printVector(double x0, double x1, double y0, double y1) {
  cout << "(" << x0 << "," << y0 << ") -> ("
       << x1 << "," << y1 << ")" << endl;
}

int main() {
  double xStart = 1.2;
  double xEnd = 2.0;
  double yStart = 0.4;
  double yEnd = 1.6;
  printVector(xStart, xEnd, yStart, yEnd);
  // (1.2,2.0) -> (0.4,1.6)
}
```

```
void offsetVector(double &x0, double &x1, double &y0, double &y1,
                  double offsetX, double offsetY) {
  x0 += offsetX;
  x1 += offsetX;
  y0 += offsetY;
  y1 += offsetY;
}

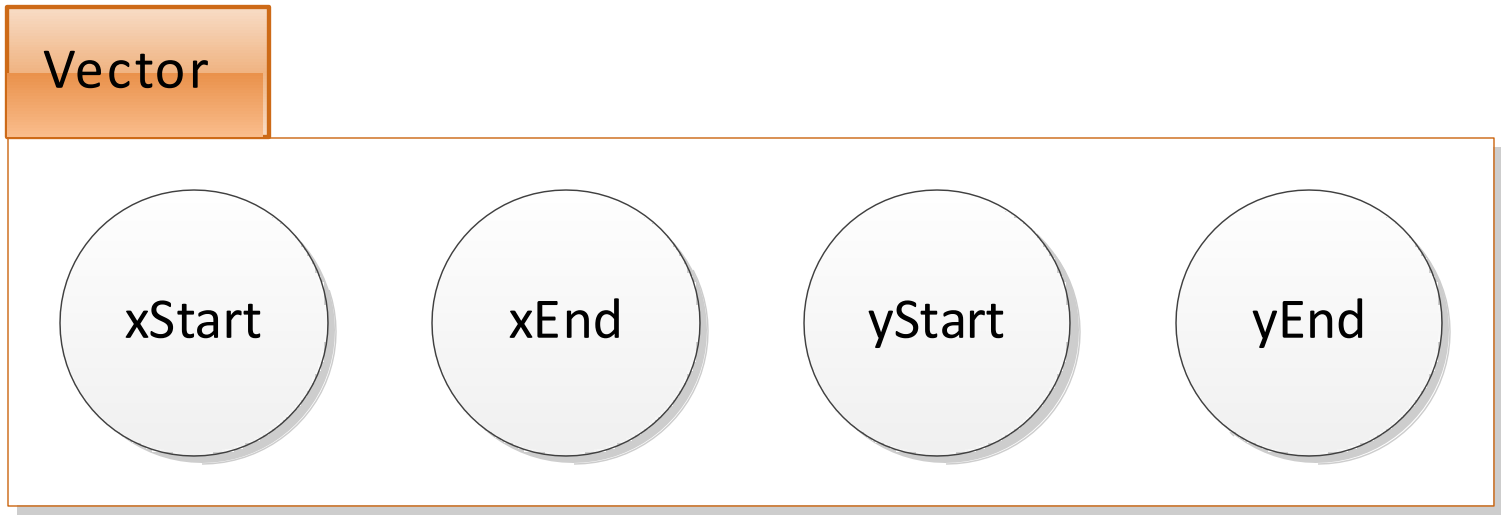
void printVector(double x0, double x1, double y0, double y1) {
  cout << "(" << x0 << "," << y0 << ") -> ("
       << x1 << "," << y1 << ")" << endl;
}

int main() {
  double xStart = 1.2;
  double xEnd = 2.0;
  double yStart = 0.4;
  double yEnd = 1.6;
  offsetVector(xStart, xEnd, yStart, yEnd, 1.0, 1.5);
  printVector(xStart, xEnd, yStart, yEnd);
  // (2.2,1.9) -> (3.0,3.1)
}
```

Many variables being passed to functions

class

- A user-defined datatype which groups together related pieces of information



class definition syntax

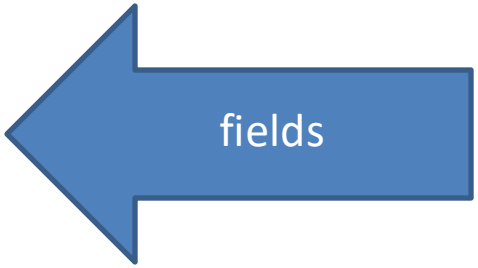



```
class Vector {
public:
  double xStart;
  double xEnd;
  double yStart;
  double yEnd;
};
```

- This indicates that the new datatype we're defining is called Vector

class definition syntax

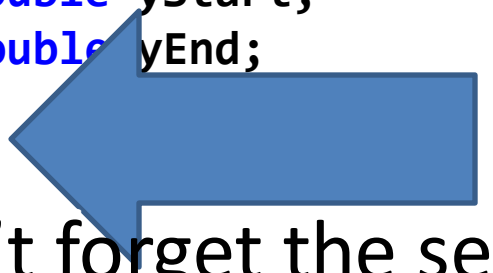
```
class Vector {
public:
  double xStart;
  double xEnd;
  double yStart;
  double yEnd;
};
```



- **Fields** indicate what related pieces of information our datatype consists of
  - Another word for field is **members**

class definition syntax

```
class Vector {
public:
  double xStart;
  double xEnd;
  double yStart;
  double yEnd;
};
```



- **NOTE:** Don't forget the semi colon!! Can cause cryptic errors if you forget it.

Fields can have different types

```
class Vector {
public:
  double xStart;
  double xEnd;
  double yStart;
  double yEnd;
  int color;

};
```

Instances

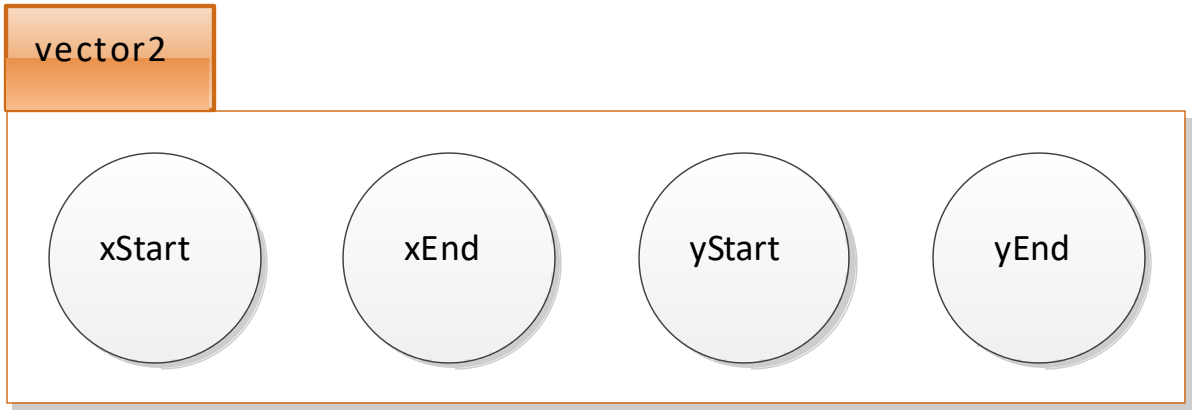
- An instance is an occurrence of a class. Different instances can have their own set of values in their fields.
- If you wanted to represent 2 different vectors you would use 2 instances of Vector

Declaring an Instance

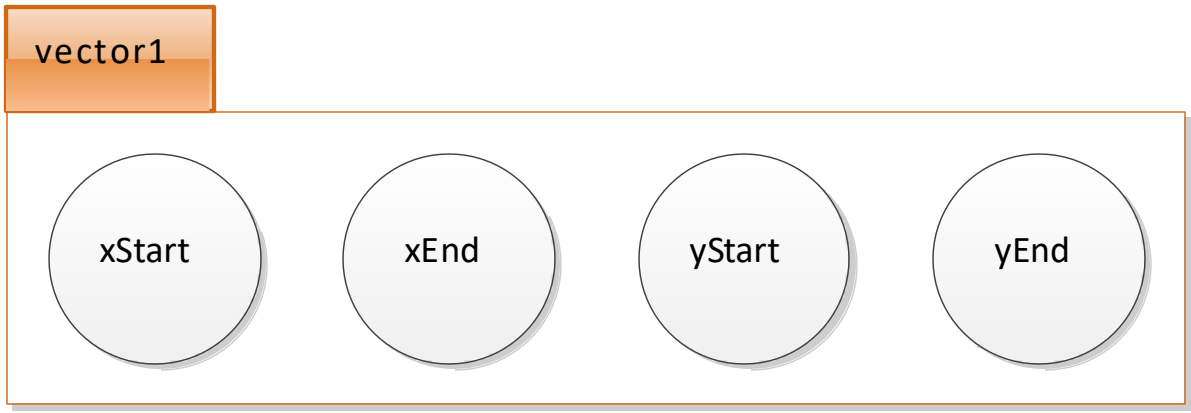
Defines 2 instances of Vector: one called vector1, the other called vector2

```
class Vector {
public:
  double xStart;
  double xEnd;
  double yStart;
  double yEnd;

};
```




```
int main() {
  Vector vector1;
  Vector vector2;
}
```

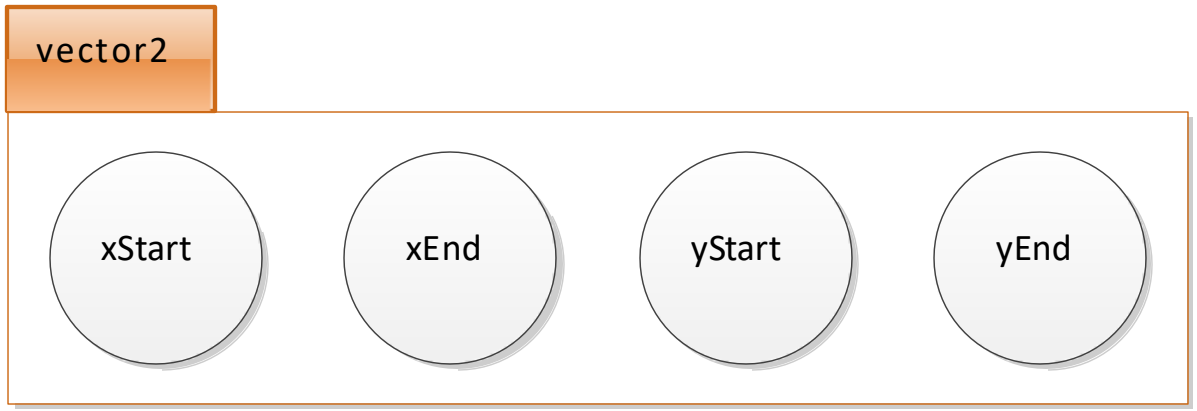


Accessing Fields

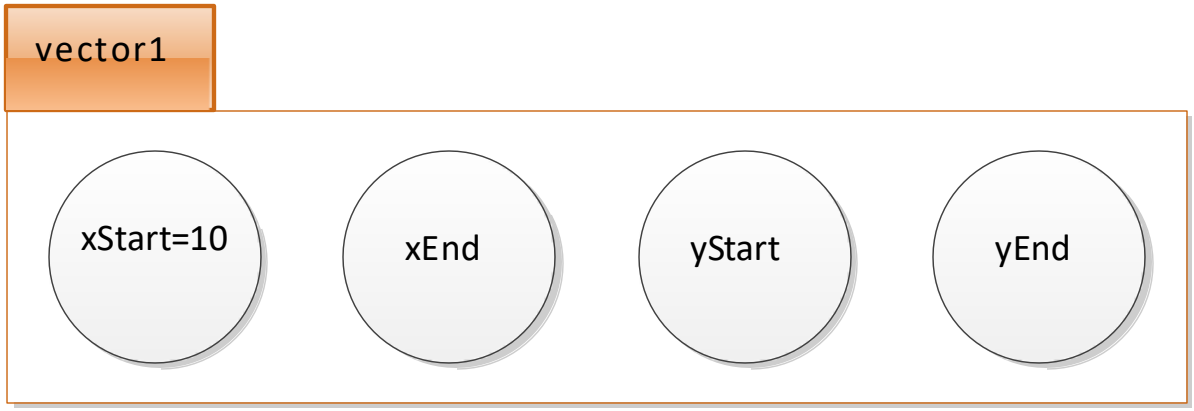
- To access fields of instances, use variable.fieldName

```
class Vector {
public:
  double xStart;
  double xEnd;
  double yStart;
  double yEnd;

};
```




```
int main() {
  Vector vector1;
  Vector vector2;
  vector2.xStart=10;
  vector1.xEnd=12;
}
```

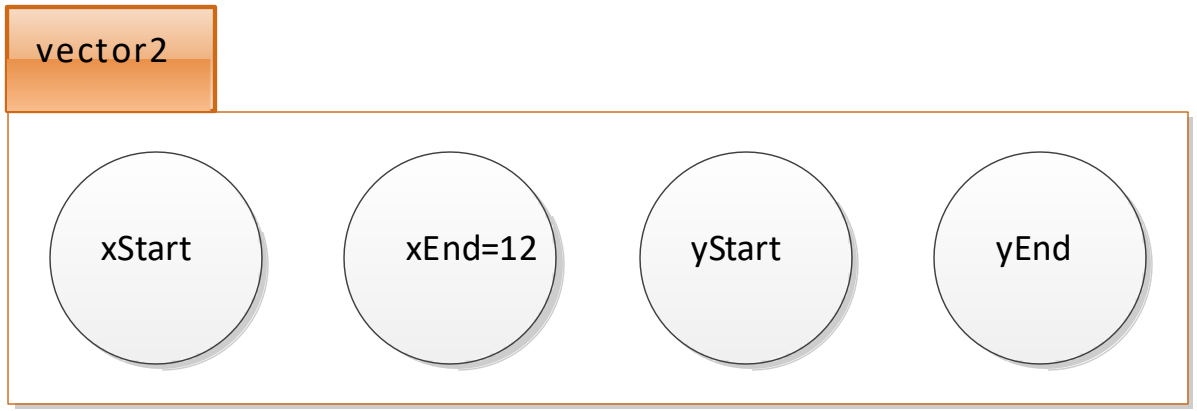


Accessing Fields

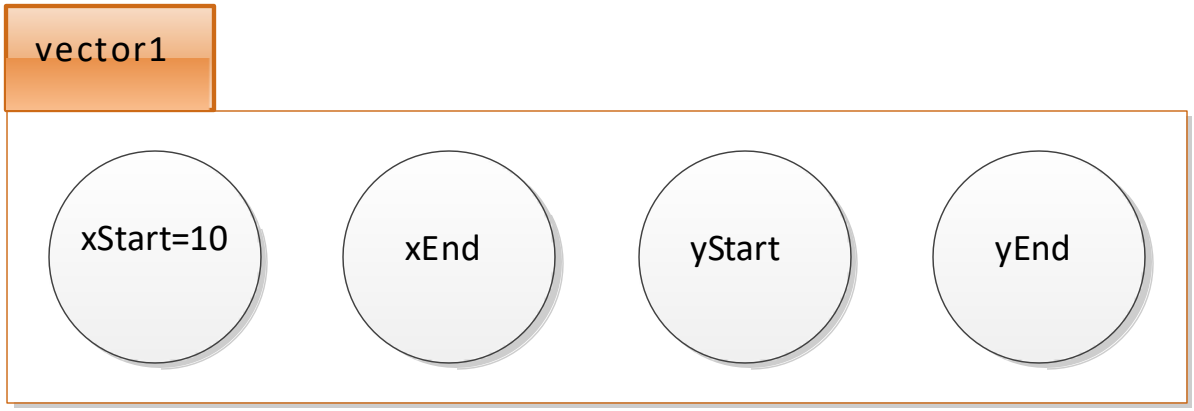
- To access fields of instances, use variable.fieldName

```
class Vector {
public:
  double xStart;
  double xEnd;
  double yStart;
  double yEnd;

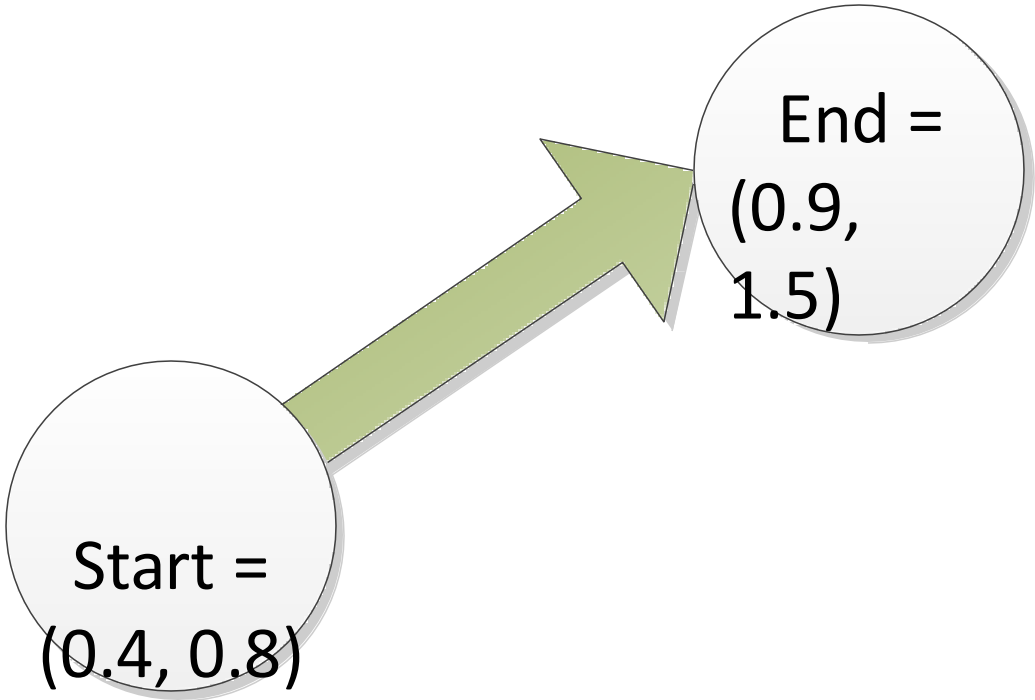
};
```




```
int main() {
  Vector vector1;
  Vector vector2;
  vector2.xStart=10;
  vector1.xEnd=12
}
```

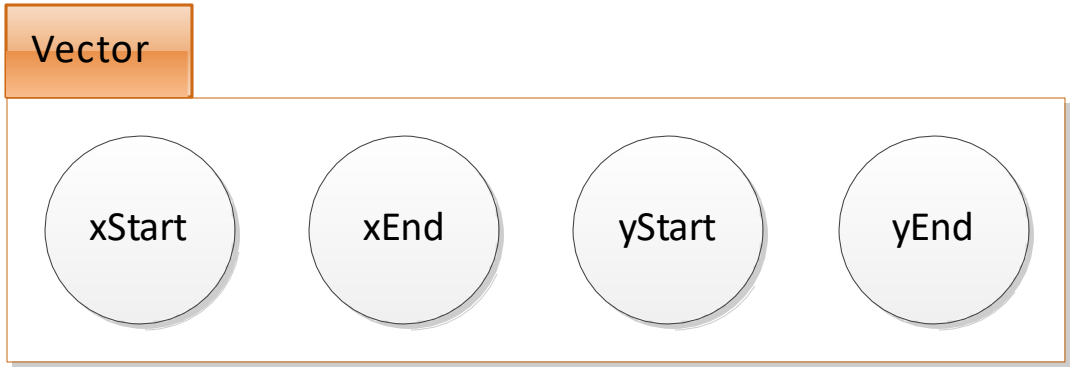


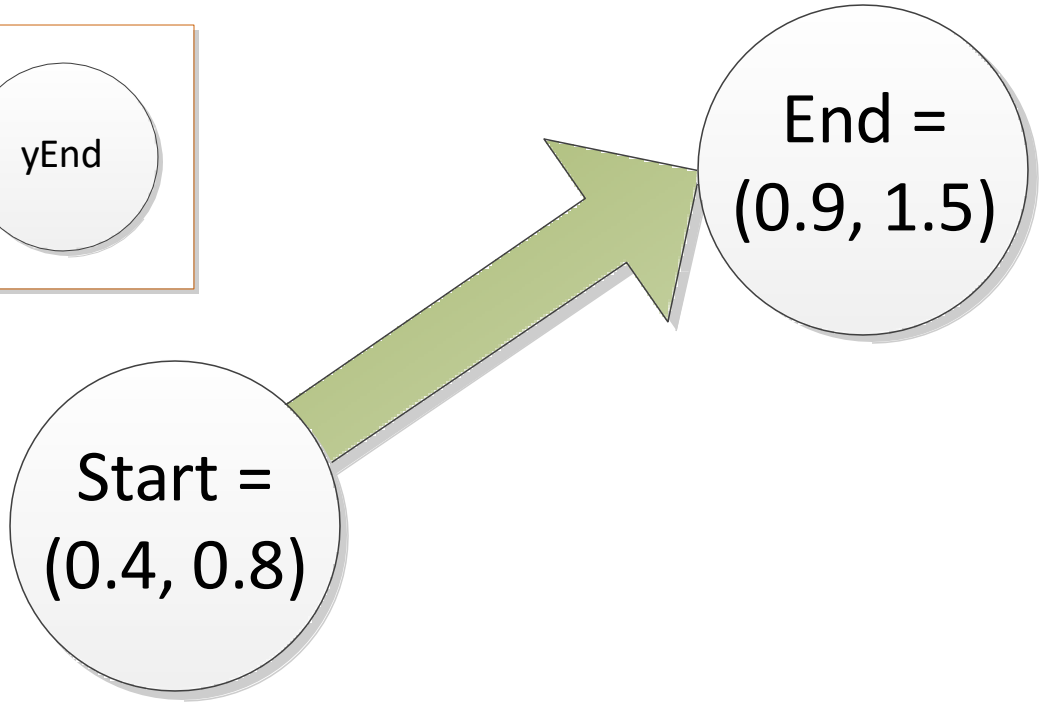
- A point consists of an x and y coordinate
- A vector consists of 2 points: a start and a finish



```
class Vector {
public:
  double xStart;
  double xEnd;
  double yStart;
  double yEnd;
};
```

- A point consists of an x and y coordinate
- A vector consists of 2 points: a start and a finish

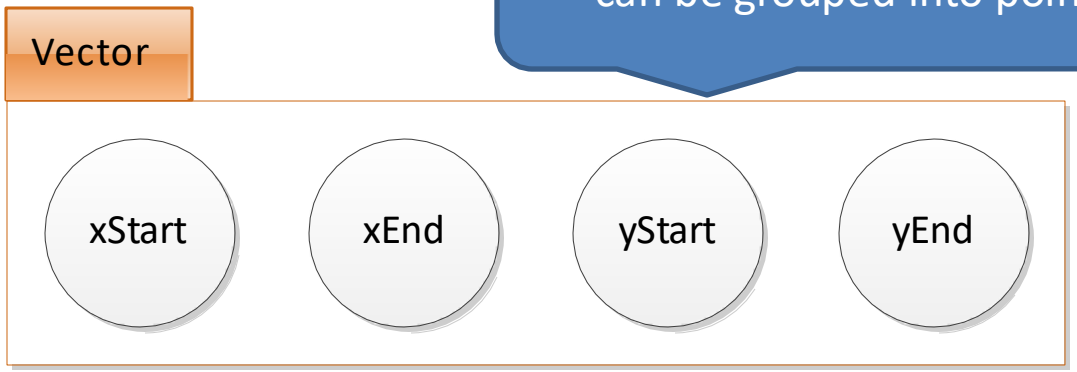


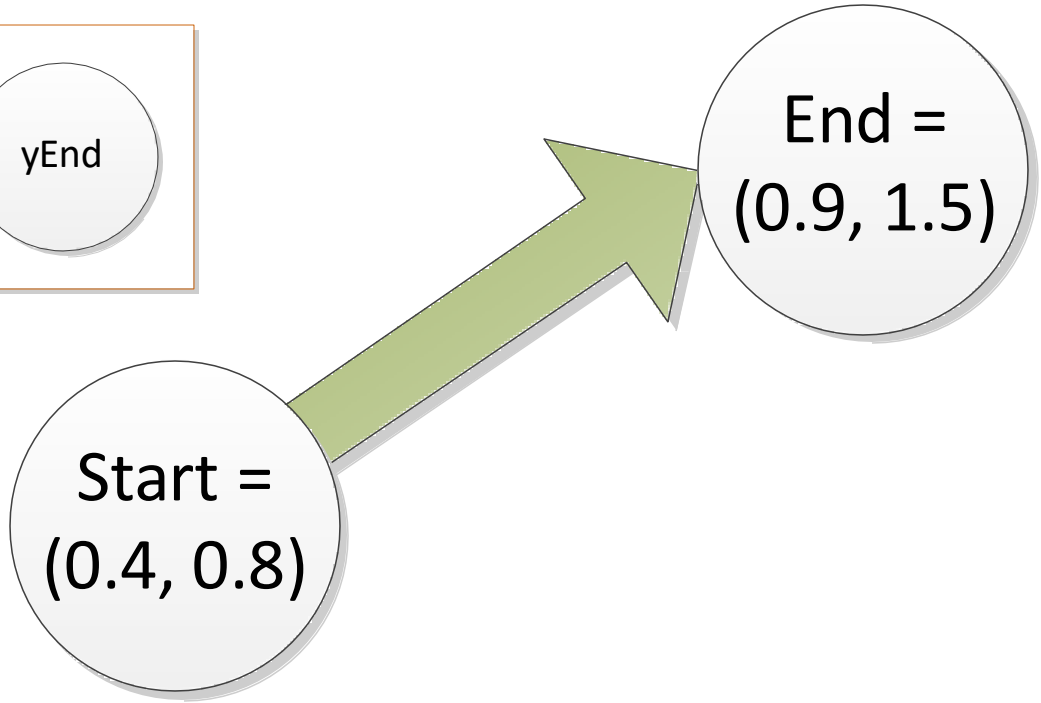


- A point consists of an x and y coordinate
- A vector consists of 2 points: a start and a finish

```
class Vector {
public:
  double xStart;
  double xEnd;
  double yStart;
  double yEnd;
};
```

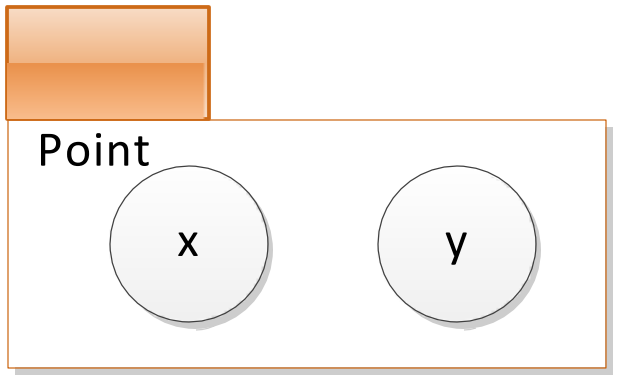
Doesn't show that coordinates can be grouped into points

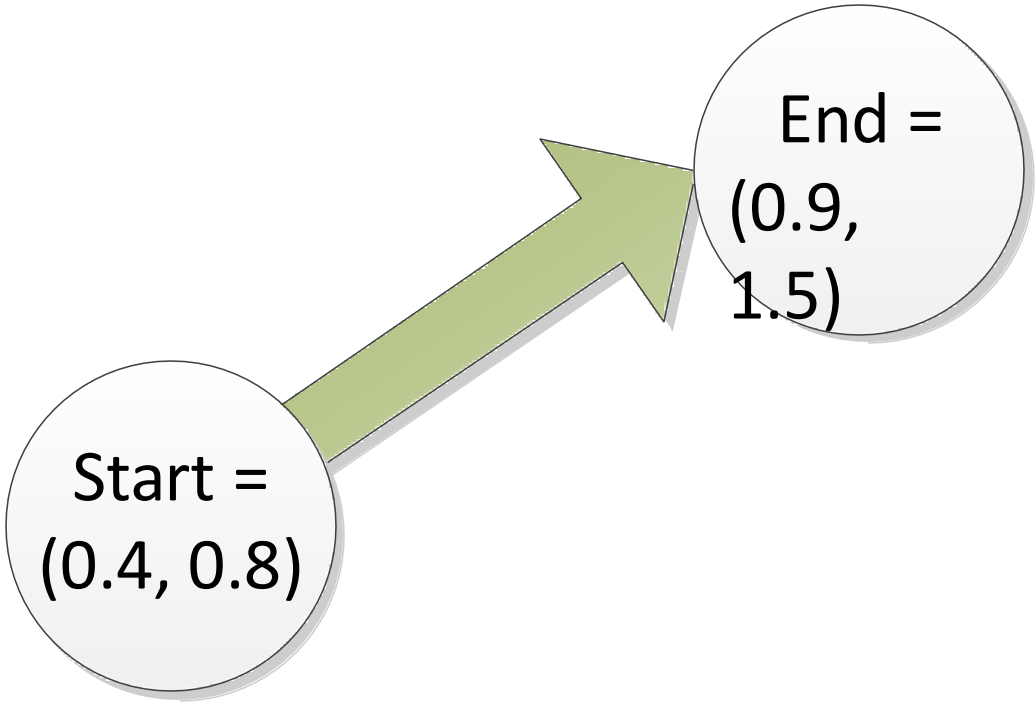




```
class Point {
public:
  double x;
  double y;
};
```

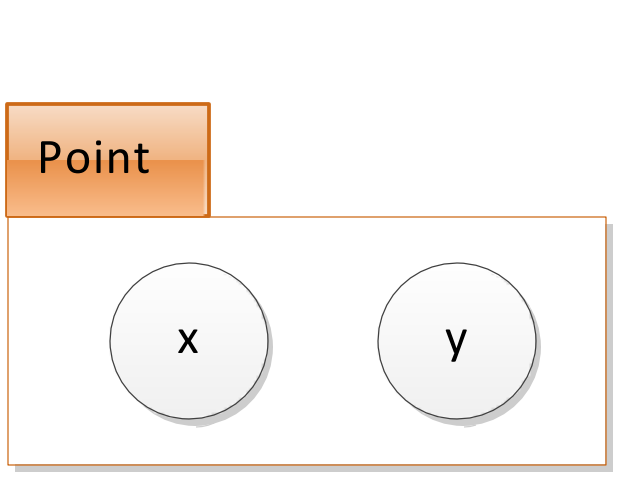
- **A point consists of an x and y coordinate**
- A vector consists of 2 points: a start and a finish

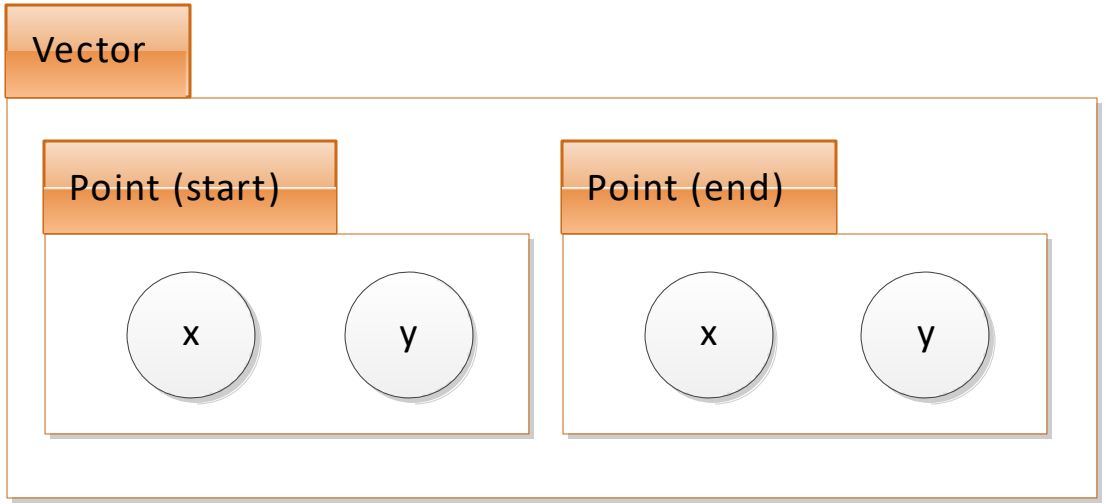




```
class Point {
public:
  double x;
  double y;
};
```

- A point consists of an x and y coordinate
- **A vector consists of 2 points: a start and a finish**



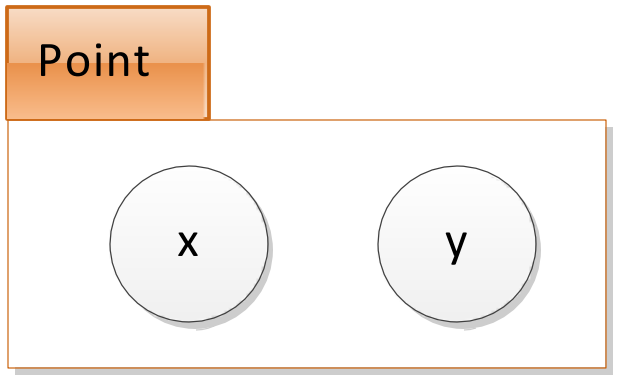


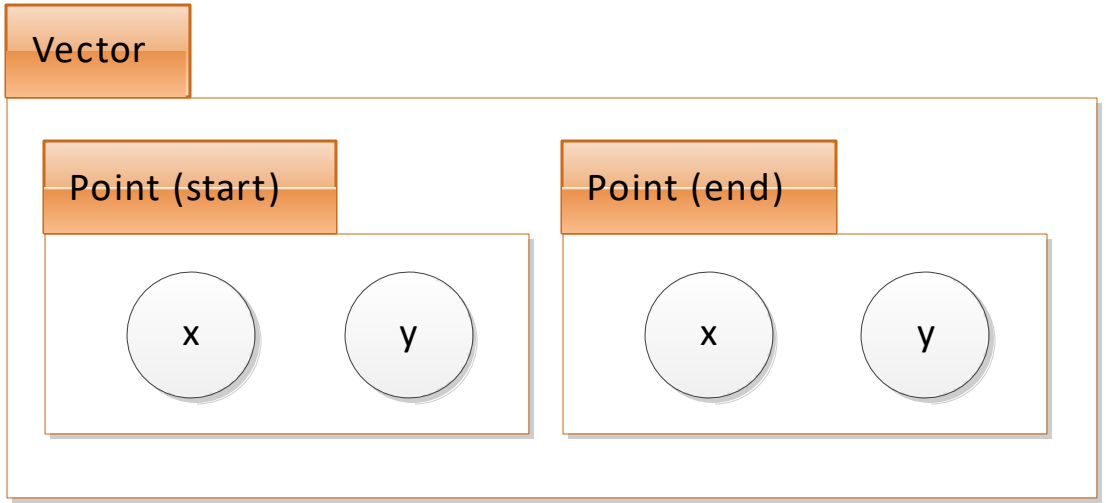
- A point consists of an x and y coordinate
- **A vector consists of 2 points: a start and a finish**

```
class Point {
public:
  double x;
  double y;
};

class Vector {
public:
  Point start;
  Point end;
};
```

Fields can be classes

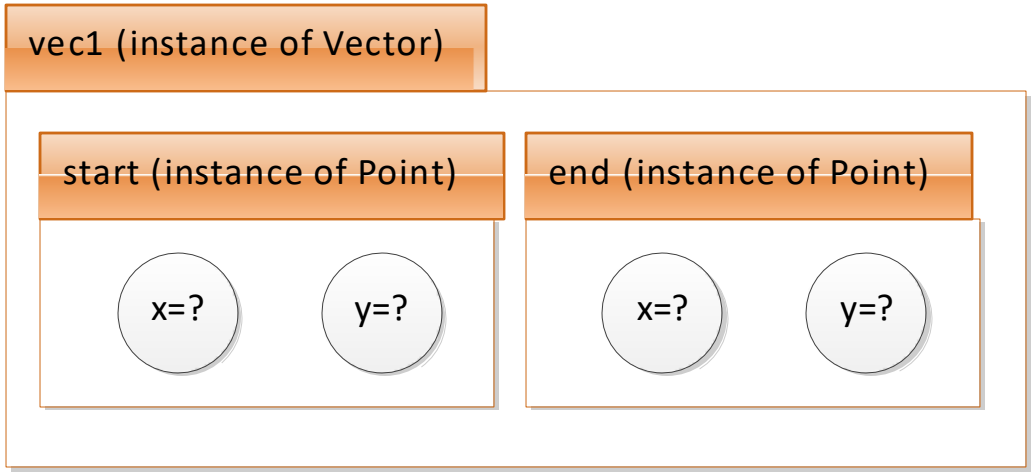




```
class Point {
public:
  double x, y;
};

class Vector {
public:
  Point start, end;
};

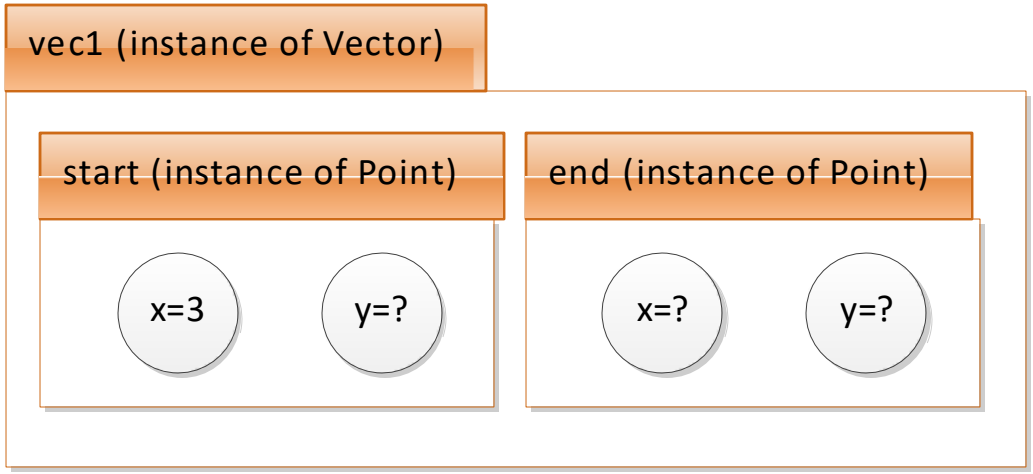
int main() {
  Vector vec1;
}
```



```
class Point {
public:
  double x, y;
};

class Vector {
public:
  Point start, end;
};

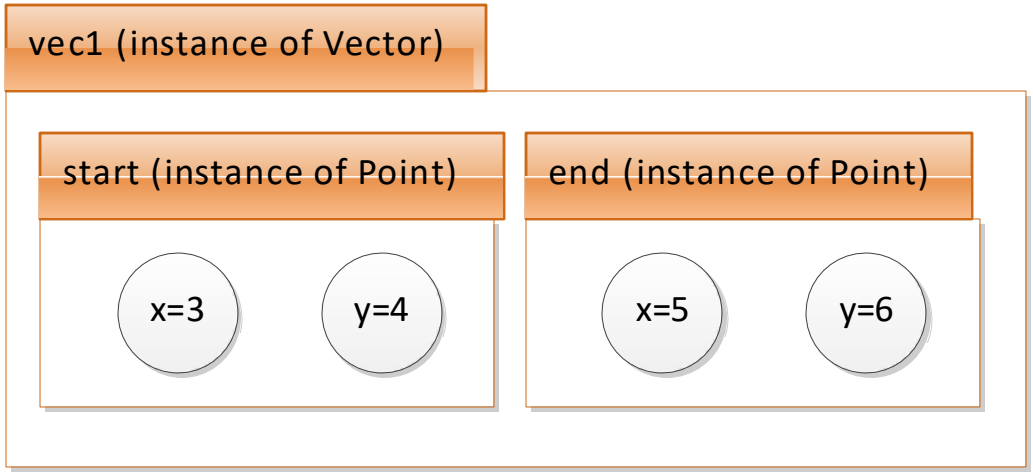
int main() {
  Vector vec1;
  vec1.start.x = 3.0;
}
```



```
class Point {
public:
  double x, y;
};

class Vector {
public:
  Point start, end;
};

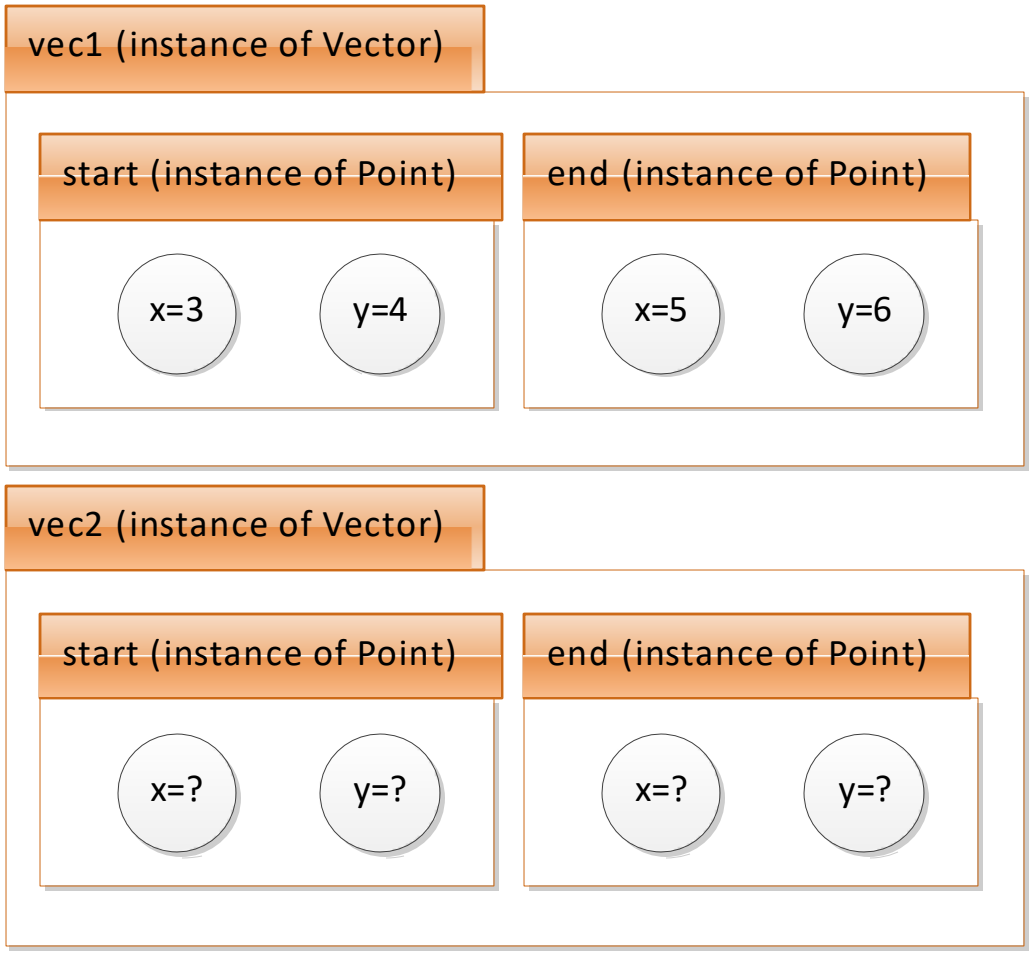
int main() {
  Vector vec1;
  vec1.start.x = 3.0;
  vec1.start.y = 4.0;
  vec1.end.x = 5.0;
  vec1.end.y = 6.0;
}
```



```
class Point {
public:
  double x, y;
};

class Vector {
public:
  Point start, end;
};

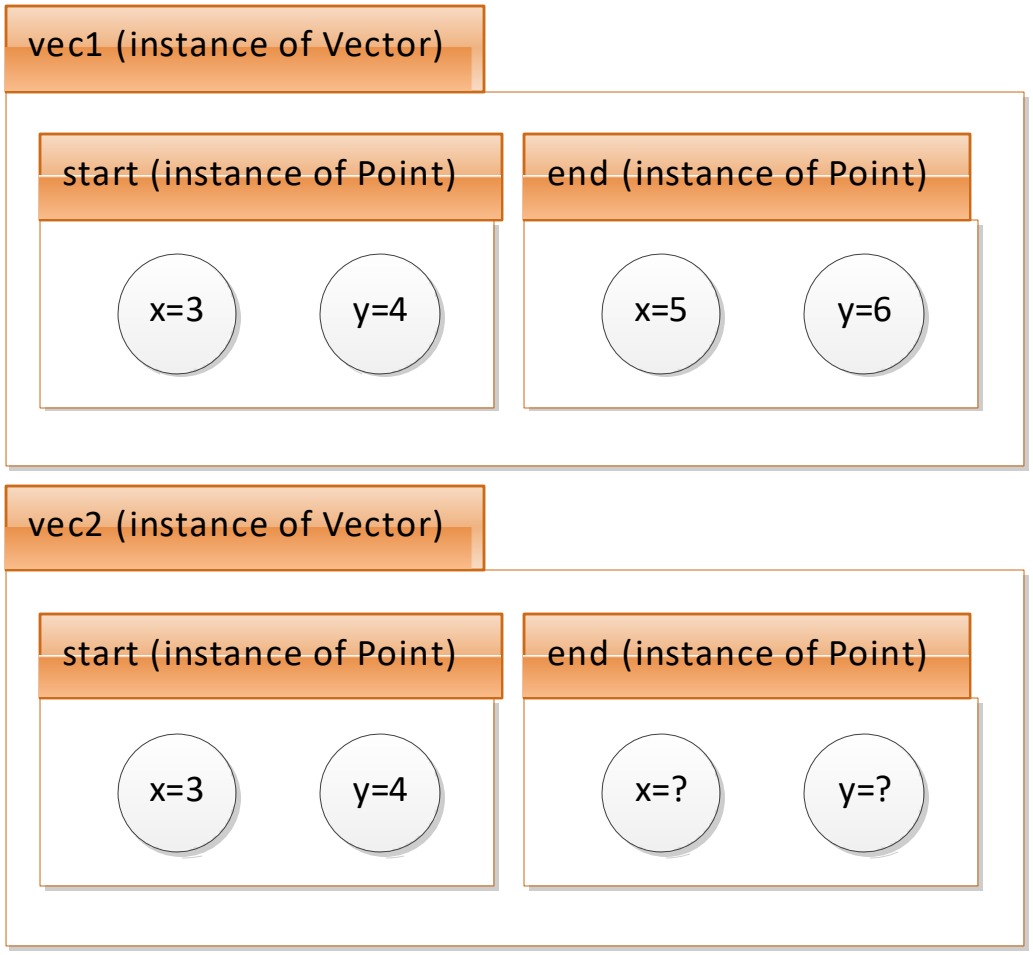
int main() {
  Vector vec1;
  vec1.start.x = 3.0;
  vec1.start.y = 4.0;
  vec1.end.x = 5.0;
  vec1.end.y = 6.0;
  Vector vec2;
}
```



```
class Point {
public:
  double x, y;
};

class Vector {
public:
  Point start, end;
};

int main() {
  Vector vec1;
  vec1.start.x = 3.0;
  vec1.start.y = 4.0;
  vec1.end.x = 5.0;
  vec1.end.y = 6.0;
  Vector vec2;
  vec2.start = vec1.start;
}
```

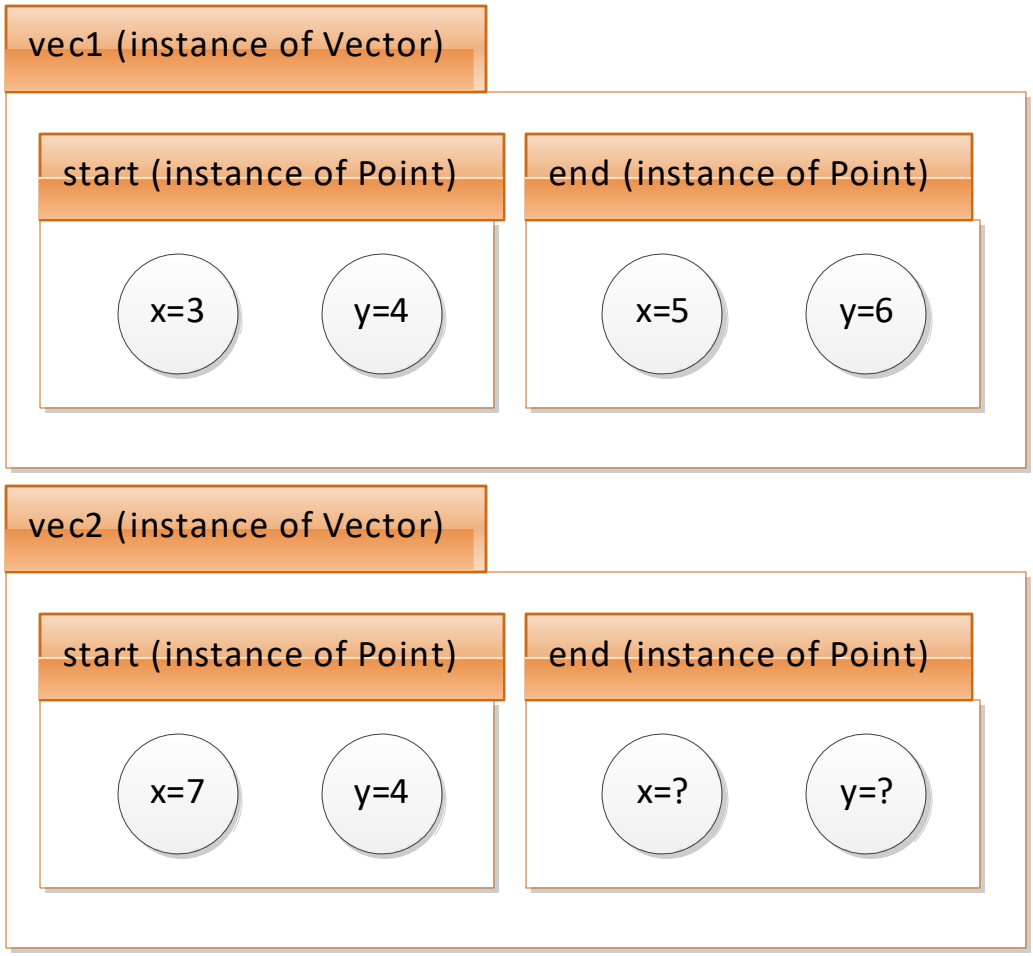


- Assigning one instance to another copies all fields

```
class Point {
public:
  double x, y;
};

class Vector {
public:
  Point start, end;
};

int main() {
  Vector vec1;
  vec1.start.x = 3.0;
  vec1.start.y = 4.0;
  vec1.end.x = 5.0;
  vec1.end.y = 6.0;
  Vector vec2;
  vec2.start = vec1.start;
  vec2.start.x = 7.0;
}
```



- Assigning one instance to another copies all fields

Passing objects to functions

- Passing by value passes a copy of the class instance to the function; changes aren't preserved

```
class Point { public: double x, y; };

void offsetPoint(Point p, double x, double y) { // does nothing
  p.x += x;
  p.y += y;
}

int main() {
  Point p;
  p.x = 3.0;
  p.y = 4.0;
  offsetPoint(p, 1.0, 2.0); // does nothing
  cout << "(" << p.x << "," << p.y << ")"; // (3.0,4.0)
}
```

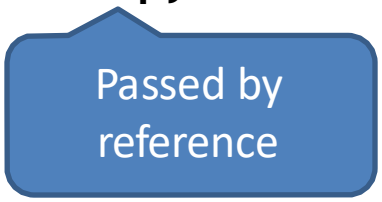
Passing objects to functions

- When a class instance is passed by reference, changes are reflected in the original

```
class Point { public: double x, y; };

void offsetPoint(Point &p, double x, double y) { // works
  p.x += x;
  p.y += y;
}

int main() {
  Point p;
  p.x = 3.0;
  p.y = 4.0;
  offsetPoint(p, 1.0, 2.0); // works
  cout << "(" << p.x << "," << p.y << ")"; // (4.0,6.0)
}
```

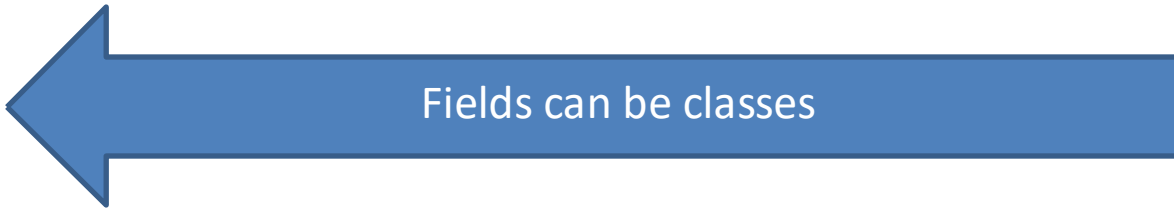


Summary

```
class Point {
  public: double x, y;
};
```

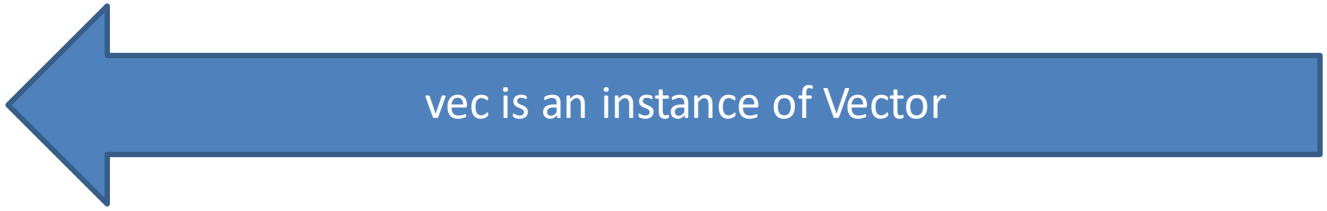


```
class Point {
  public: double x, y;
};
class Vector {
  public: Point start, end;
};
```



```
class Point {
  public: double x, y;
};
class Vector {
  public: Point start, end;
};
```

```
int main() {
  Vector vec;
}
```



```
class Point {
  public: double x, y;
};
class Vector {
  public: Point start, end;
};
```

```
int main() {
  Vector vec;
  vec.start.x = 1.2;
}
```



```
class Point {
  public: double x, y;
};
class Vector {
  public: Point start, end;
};
```

```
int main() {
  Vector vec;
  vec.start.x = 1.2; vec.end.x = 2.0; vec.start.y = 0.4; vec.end.y = 1.6
}
```

```
class Point {
  public: double x, y;
};
class Vector {
  public: Point start, end;
};
```

```
void printVector(Vector v) {
  cout << "(" << v.start.x << "," << v.start.y << ") -> (" << v.end.x <<
"," << v.end.y << ")" << endl;
}

int main() {
  Vector vec;
  vec.start.x = 1.2; vec.end.x = 2.0; vec.start.y = 0.4; vec.end.y = 1.6
  printVector(vec); // (1.2,0.4) -> (2.0,1.6)
}
```

Objects can be passed to functions

```
class Point {
  public: double x, y;
};
class Vector {
  public: Point start, end;
};
```

Can pass to value if you don't need to modify the class

```
void printVector(Vector v) {
  cout << "(" << v.start.x << "," << v.start.y << ") -> (" << v.end.x <<
"," << v.end.y << ")" << endl;
}

int main() {
  Vector vec;
  vec.start.x = 1.2; vec.end.x = 2.0; vec.start.y = 0.4; vec.end.y = 1.6
  printVector(vec); // (1.2,0.4) -> (2.0,1.6)
}
```

```
class Point {
  public: double x, y;
};
class Vector {
  public: Point start, end;
};
```

Pass objects by reference if they need to be modified

```
void offsetVector(Vector &v, double offsetX, double offsetY) {
  v.start.x += offsetX;
  v.end.x += offsetX;
  v.start.y += offsetY;
  v.end.y += offsetY;
}
void printVector(Vector v) {
  cout << "(" << v.start.x << "," << v.start.y << ") -> (" << v.end.x <<
"," << v.end.y << ")" << endl;
}

int main() {
  Vector vec;
  vec.start.x = 1.2; vec.end.x = 2.0; vec.start.y = 0.4; vec.end.y = 1.6
  offsetVector(vec,  1.0, 1.5);
  printVector(vec); // (2.2,1.9) -> (3.8,4.3)
}
```

```
class Point {
  public: double x, y;
};
class Vector {
  public: Point start, end;
};
```

Pass objects by pointer if they need to be modified

```
void offsetVector(Vector *v, double offsetX, double offsetY) {
  v->start.x +=offsetX;
  v->end.x +=offsetX;
  v->start.y +=offsetY;
  v->end.y += offsetY;
}
void printVector(Vector v) {
  cout << "(" << v.start.x << "," << v.start.y << ") -> (" << v.end.x <<
"," << v.end.y << ")" << endl;
}

int main() {
  Vector vec;
  vec.start.x = 1.2; vec.end.x = 2.0; vec.start.y = 0.4; vec.end.y = 1.6
  offsetVector(&vec, 1.0, 1.5);
  printVector(vec); // (2.2,1.9) -> (3.8,4.3)
}
```

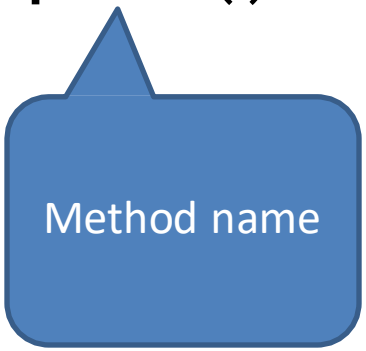
- Observe how some functions are closely associated with a particular class

```
void offsetVector(Vector &v, double offsetX, double offsetY);
void printVector(Vector v);
```

```
int main() {
  Vector vec;
  vec.start.x = 1.2; vec.end.x = 2.0;
  vec.start.y = 0.4; vec.end.y = 1.6;
  offsetVector(vec,  1.0, 1.5);
  printVector(vec);
}
```

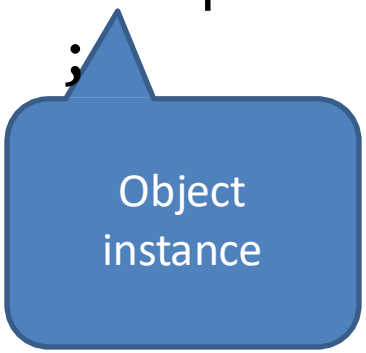
- Observe how some functions are closely associated with a particular class
- **Methods**: functions which are part of a class

```
Vector vec;
vec.start.x = 1.2; vec.end.x = 2.0;
vec.start.y = 0.4; vec.end.y = 1.6;
vec.print()
;
```



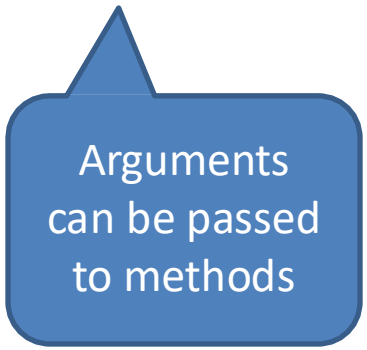
- Observe how some functions are closely associated with a particular class
- **Methods**: functions which are part of a class
  - Implicitly pass the current instance

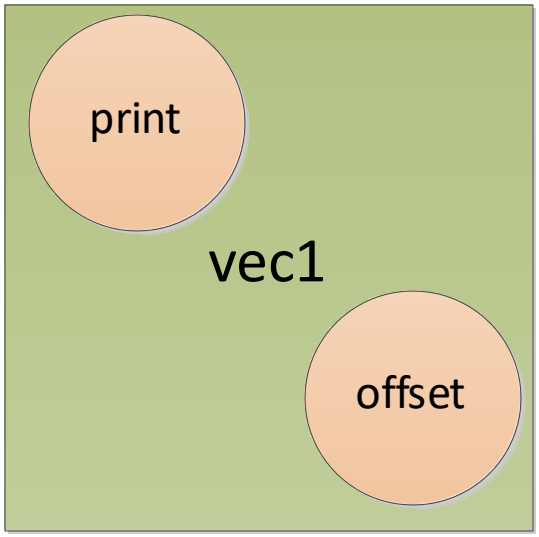
```
Vector vec;
vec.start.x = 1.2; vec.end.x = 2.0;
vec.start.y = 0.4; vec.end.y = 1.6;
vec.print()
;
```

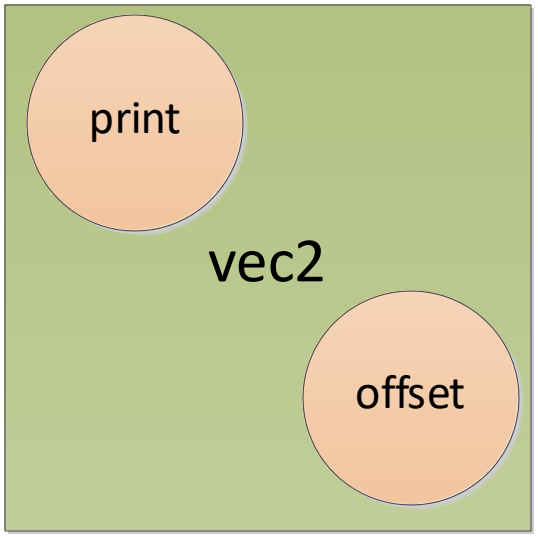


- Observe how some functions are closely associated with a particular class
- **Methods**: functions which are part of a class
  - Implicitly pass the current instance

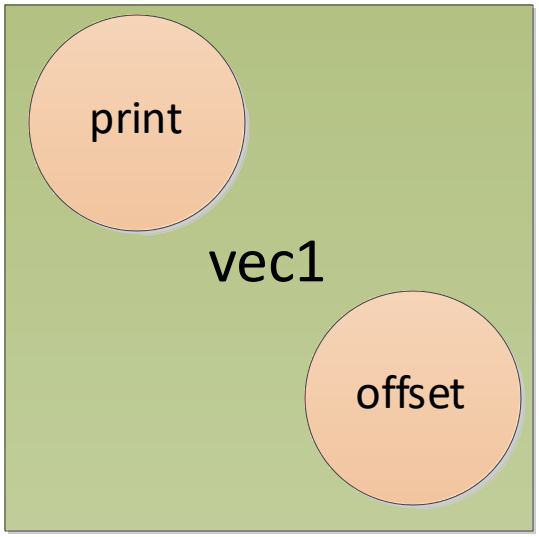
```
Vector vec;
vec.start.x = 1.2; vec.end.x = 2.0;
vec.start.y = 0.4; vec.end.y = 1.6;
vec.print();
vec.offset(1.0, 1.5);
```

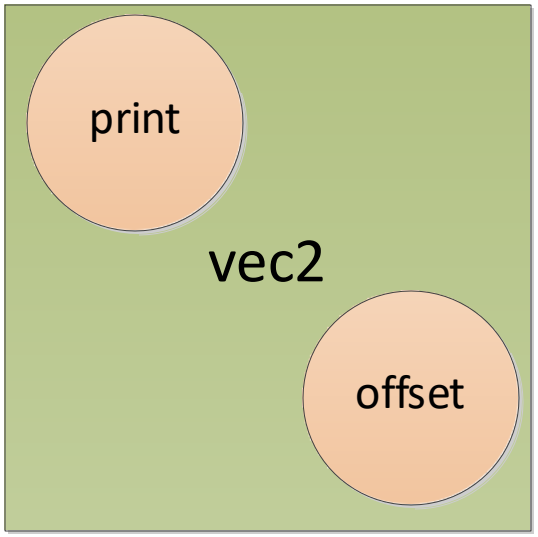




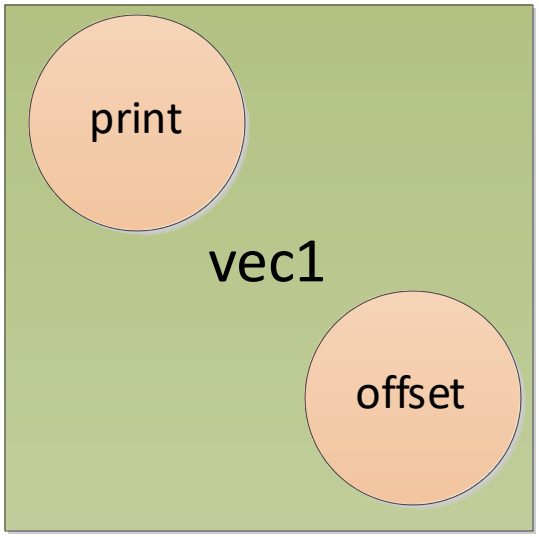


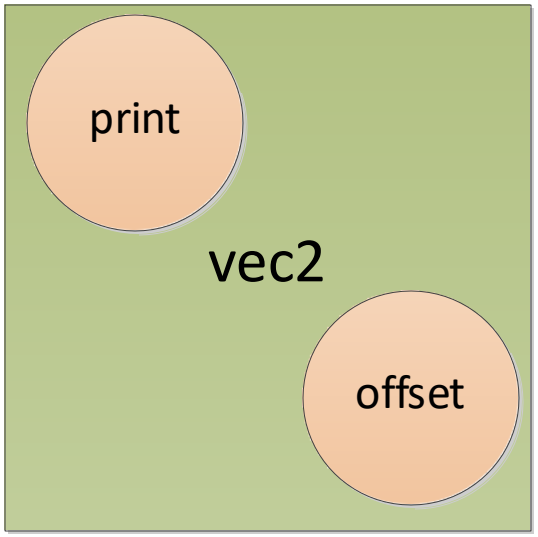

```
Vector vec1;
Vector vec2;
// initialize vec1 and vec2
vec1.print();
```






```
Vector vec1;
Vector vec2;
// initialize vec1 and vec2
vec1.print();
```

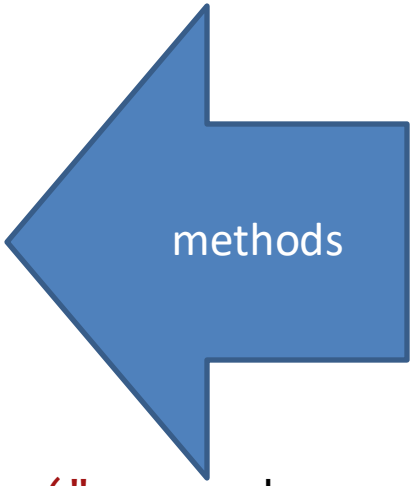





```
Vector vec1;
Vector vec2;
// initialize vec1 and vec2
vec1.print();
```

```
class Vector {
public:
  Point start;
  Point end;

  void offset(double offsetX, double offsetY) {
    start.x += offsetX;
    end.x += offsetX;
    start.y += offsetY;
    end.y += offsetY;
  }
  void print() {
    cout << "(" << start.x << "," << start.y << ") -> (" << end.x <<
"," << end.y << ")" << endl;
  }
};
```



```
class Vector {
public:
  Point start;
  Point end;

  void offset(double offsetX, double offsetY) {
    start.x += offsetX;
    end.x += offsetX;
    start.y += offsetY;
    end.y += offsetY;
  }
  void print() {
    cout << "(" << start.x << "," << start.y << ") -> (" << end.x <<
"," << end.y << ")" << endl;
  }
};
```



```
class Vector {
public:
  Point start, end;

  void offset(double offsetX, double offsetY) {
    start.offset(offsetX, offsetY);
    end.offset(offsetX, offsetY);
  }
  void print() {
    start.print();
    cout << " -> ";
    end.print();
    cout << endl;
  }
};
```




```
class Point {
public:
  double x, y;
  void offset(double offsetX, double offsetY) {
    x += offsetX; y += offsetY;
  }
  void print() {
    cout << "(" << x << "," << y << ")";
  }
};
```

Implementing Methods Separately

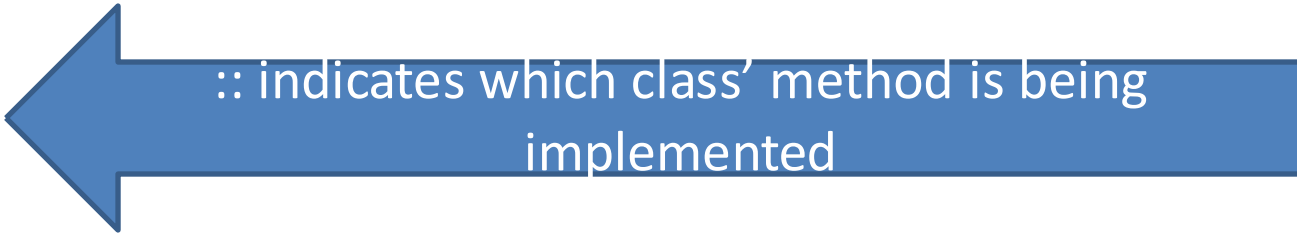
- Recall that function prototypes allowed us to declare that functions will be implemented later
- This can be done similarly for class methods

```
// vector.h - header file
class Point {
public:

  double x, y;
  void offset(double offsetX, double offsetY);
  void print();
};

class Vector {
public:
  Point start, end;
  void offset(double offsetX, double offsetY);
  void print();
};
```

```
#include "vector.h"
// vector.cpp - method implementation
void Point::offset(double offsetX, double offsetY) {
  x += offsetX; y += offsetY;
}
void Point::print() {
  cout << "(" << x << "," << y << ")";
}
void Vector::offset(double offsetX, double offsetY) {
  start.offset(offsetX, offsetY);
  end.offset(offsetX, offsetY);
}
void Vector::print() {
  start.print();
  cout << " -> ";
  end.print();
  cout << endl;
}
```



- Manually initializing your fields can get tedious
- Can we initialize them when we create an instance?

```
Vector vec;
vec.start.x = 0.0;
vec.start.y = 0.0;
vec.end.x = 0.0;
vec.end.y = 0.0;
```

```
Point p;
p.x = 0.0;
p.y = 0.0;
```

Constructors

- Method that is called when an instance is created

```
class Point {
public:
  double x, y;
  Point() {
    x = 0.0; y = 0.0; cout << "Point instance created" << endl;
  }
};

int main() {
  Point p; // Point instance created
  // p.x is 0.0, p.y is 0.0
}
```

Constructors

- Can accept parameters

```
class Point {
public:
  double x, y;
   Point() {
    x = 0.0; y = 0.0; cout << "Point instance created" <<
    endl;
  }

  Point(double nx, double ny) {
    x = nx; y = ny; cout << "2-parameter constructor" << endl;
  }
};

int main() {
  Point p(2.0, 3.0); // 2-parameter constructor
  // p.x is 2.0, p.y is 3.0
}
```

Constructors

- Can have multiple constructors

```
class Point {
public:
  double x, y;
  Point() {
    x = 0.0; y = 0.0; cout << "default constructor" << endl;
  }
  Point(double nx, double ny) {
    x = nx; y = ny; cout << "2-parameter constructor" << endl;
  }
};

int main() {
  Point p; // default constructor
  // p.x is 0.0, p.y is 0.0)
  Point q(2.0, 3.0); // 2-parameter constructor
  // q.x is 2.0, q.y is 3.0)
}
```

Initialization list

```
class Point {
public:
  double x, y;
  Point():x(0)
  {
    y = 0.0; cout << "default constructor" << endl;
  }
  // use either
  Point(double nx, double ny):x(nx),y{ny} {
  }
};

int main() {
  Point p; // default constructor

  // p.x is 0.0, p.y is 0.0)
  Point q(2.0, 3.0); // 2-parameter constructor
  // q.x is 2.0, q.y is 3.0)
}
```

Using a pointer to an object

```
int main() {
  Point* pPtr; // Pointer to Point object. No instance created.
  Point p;
  // p.x is 0.0, p.y is 0.0
  pPtr = &p;
  cout << p.x << " " << p.y;
  cout << pPtr->x << " " << pPtr->y;
  pPtr->x=10;
  p.y=20;
  // use the '->' to access the member data/methods when the
  variable points to an instance

  cout << p.x << " " << p.y;
  cout << pPtr->x << " " << pPtr->y;



}
```

Using New

- Use new to create an instance

```
class Point {
public:
  double x, y;
  Point() {
    x = 0.0; y = 0.0; cout << "Point instance created" << endl;
  }
};

int main() {
  Point* p; // Pointer to Point object. No instance created.
  p = new Point(); //instance created and p points to it

  // p->x is 0.0, p->y is 0.0
  p->x =10;

  cout << p->x << “ ” << p->y;
  // use the ‘->’ to access the member data/methods when created
  using a pointe
```

Using a pointer to an object created with new

```
int main() {
  Point* p; // Pointer to Point object. No instance created.
  p = new Point();
  Point* pPtr;
  pPtr = p;
  cout << p.x << " " << p.y;
  cout << pPtr->x << " " << pPtr->y;
  pPtr->x=10;
  p.y=20;
  cout << p.x << " " << p.y;
  cout << pPtr->x << " " << pPtr->y;


}
```

Using a pointer to an object created with new

```
int main() {
  Point* p; // Pointer to Point object. No instance created.
  p = new Point();
  Point* pPtr;
  pPtr = p;
  cout << p->x << " " << p->y;
  cout << pPtr->x << " " << pPtr->y;
  pPtr->x=10;
  P->y=20;
  cout << p->x << " " << p->y;
  cout << pPtr->x << " " << pPtr->y;


}
```

- The “this” pointer

```
class Point {
public:
  double x, y;
  Point() {
    this->x = 0.0; this->y = 0.0;
    cout << "default constructor" << endl;
  }
  Point(double nx, double ny) {
    x = nx; y = ny; cout << "2-parameter constructor" << endl;
  }
};

int main() {
  Point q(1.0, 2.0); // 2-parameter constructor
  // r.x is 1.0, r.y is 2.0)
}
```

- Static members

```
class Maths {
public:
        static float PI;
        static float Mul(Point a, Point b)
        {
                //cannot access any non static member
                // of Maths
                return  a.x * b.x + a.y * b.y;
        }

};


int main() {
Maths::PI = 3.14; // use 2 colons to access static
Point q(1.0, 2.0);
float result =  Maths::Mul(q, q);
}
```

Static member and non static members

```
class Maths {
public:
        int temp; // non static
        static float PI;
        static float Mul(Point a, Point b)
        {
                this->temp = 10; // cannot access
        not static members. Compile error
                 temp = 10; // cannot access        not
static members. Compile error

                return  a.x * b.x + a.y * b.y;
        }

};
```

- You can define your own copy constructor

```
class Point {
public:
  double x, y;
  Point(double nx, double ny) {
    x = nx; y = ny; cout << "2-parameter constructor" << endl;
  }
  // custom copy constructor. See later for better example
  Point(Point &o) {
    x = o.x; y = o.y; cout << "custom copy constructor" << endl;
  }
};

int main() {
  Point q(1.0, 2.0); // 2-parameter constructor
  Point r = q; // custom copy constructor
  // r.x is 1, r.y is 2
}
```

Access Modifiers

- Define where your fields/methods can be accessed from




```
class Point {
public:
  double x, y;

  Point(double nx, double ny) {
    x = nx; y = ny;
  }
};
```

Access Modifiers

- public: can be accessed from anywhere

```
class Point {
public:
  double x, y;

  Point(double nx, double ny) {
    x = nx; y = ny;
  }
};

int main() {
  Point p(2.0,3.0);
  p.x = 5.0; // allowed
}
```

Access Modifiers

- private: can only be accessed within the class

```
class Point {
private:
  double x, y;

public:
  Point(double nx, double ny) {
    x = nx; y = ny;
  }
};

int main() {
  Point p(2.0,3.0);
  p.x = 5.0; // not allowed
}
```

Access Modifiers

- Use getters to allow read-only access to private fields

```
class Point {
private:
  double x, y;

public:
  Point(double nx, double ny) {
    x = nx; y = ny;
  }
  double getX() { return x; }
  double getY() { return y; }
  bool setX( double px ) { x=px;return
true;}

};

int main() {
  Point p(2.0,3.0);
  cout << p.getX() << endl; // allowed
```

Default Access Modifiers

- class: private by default

```
class Point {
  double x, y;
};
```




```
class Point {
private:
  double x, y;
};
```

Copy Constructors

- Why make a copy constructor? Assigning all fields (default copy constructor) may not be what you want

```
class MITStudent {
public:
  int studentID;
  char *name;
  MITStudent() {
    studentID =0;
    name=NULL;
  }
};
```

```
int main() {
  MITStudent student1;
  student1.studentID = 98;
  char n[] = "foo";
  student1.name = n;
  MITStudent student2 = student1;
  student2.name[0] = 'b';
  cout << student1.name; // boo
}
```



- Why make a copy constructor? Assigning all fields (default copy constructor) may not be what you want

  strdup() is a c function that duplicates a string.

```
#include <string.h>
class MITStudent {
public:
  int studentID;
  char *name;
  MITStudent() {
    studentID =0;
    name=NULL;}
  MITStudent(MITStudent &o) {
    studentID = o.studentID;
    name = _strdup(o.name);
  }
};
```

```
int main() {
  MITStudent student1;
  student1.studentID = 98;
  char n[] = "foo";
  student1.name = n;
  MITStudent student2 = student1;
  student2.name[0] = 'b';
  cout << student1.name; // foo
}
```

Structs

- Structs are a carry-over from the C; in C++, classes are generally used
- In C++, they're essentially the same as classes, except structs' default access modifier is public

```
class Point {
public:
  double x;
  double y;
};
```

```
struct Point {

  double x;
  double y;
};
```

Default Access Modifiers

- struct: public by default
- class: private by default

```
struct Point {
  double x, y;
};
```




```
struct Point {
public:
  double x, y;
};
```

```
class Point {
  double x, y;
};
```




```
class Point {
private:
  double x, y;
};
```

Arrays of Objects

```
#include <iostream>
class Point
{
public:
        int x;
        Point(int val)
                :x(val)
        {

        }
        void Print()
        {
                std::cout << x;
        }
        Point()
        {

        }
};
```

```
int main()
{
        Point points[20];
        for (int i = 0; i < 20; i++)
        {
                points[i].x = i;
        }
        for (int i = 0; i < 20; i++)
        {
                points[i].Print();
        }


}
```

Using new to create the array

```
int main()
{
        Point* points = new Point[30];
        for (int i = 0; i < 20; i++)
        {
                points[i].x = i;
        }
        for (int i = 0; i < 20; i++)
        {
                points[i].Print();
        }


        delete[] points;

}
```

Array of pointers

```
int main()
{
        Point* pointPointers[20];
        for (int i = 0; i < 20; i++)
        {
                pointPointers[i] = new Point(i);
        }
        for (int i = 0; i < 20; i++)
        {
                pointPointers[i]->Print();
        }

        for (int i = 0; i < 20; i++)
        {
                delete pointPointers[i];
        }

 }
```

Using new for the array of pointers

```
int main()
{
        Point** pointPointers = new Point*[30];
        for (int i = 0; i < 30; i++)
        {
                pointPointers[i] = new Point(i);
        }
        for (int i = 0; i < 30; i++)
        {
                pointPointers[i]->Print();
        }

        for (int i = 0; i < 30; i++)
        {
                delete pointPointers[i];
        }
        delete[] pointPointers;

}
```

Destructors

A destructor is called when an object goes out of Scope

~ClassName()

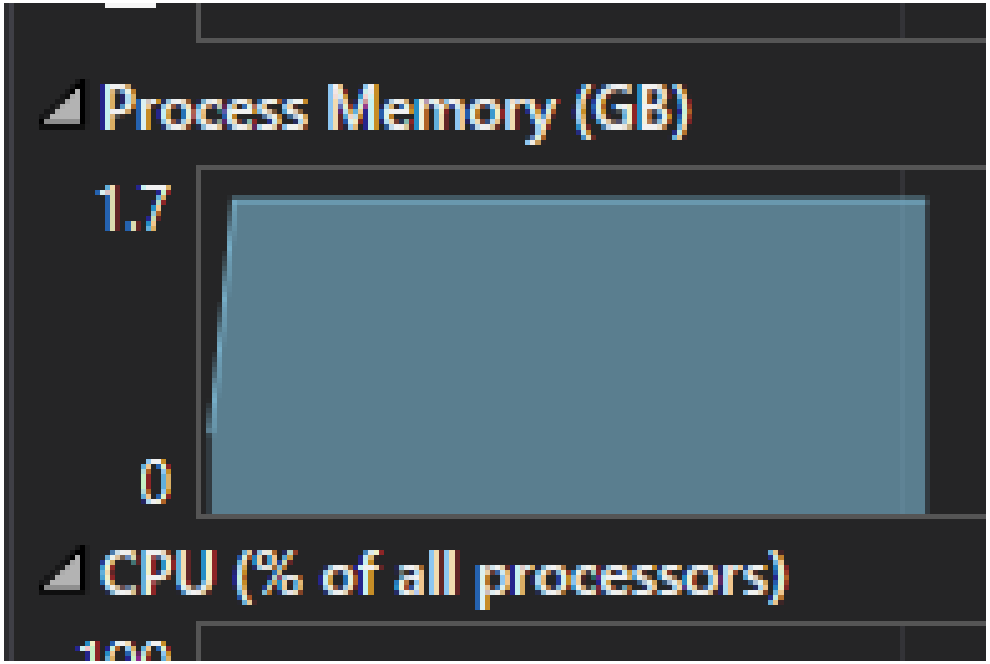
```
class Point
{
public:
int x, y;
};

class TwentyThousandPoints
{
public:
Point* points;
TwentyThousandPoints()
{
points = new Point[20000];
}
~TwentyThousandPoints()
{
delete[] points;
}
};
```

```
int main()
{
 {
    TwentyThousandPoints points[1000
 }
while (1)
{
int x = 10;
x++;
}

}
```

Without Destructor



With Destructor

